11

BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Atomik İlkel Tipler: AtomicInteger, CAS (Compare-And-Swap) & Kilit-Siz Programlama

Atomic Primitives: AtomicInteger, CAS (Compare-And-Swap) & Lock-Free Computing

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              COMPARE-AND-SWAP (CAS) DONANIMSAL DÖNGÜSÜ   │
└──────────────────────────────────────────────────────────┘

 Thread 1 ve Thread 2 aynı anda count'u (5) artırmak istiyor:

 1. Thread 1 okur: Beklenen Değer = 5, Yeni Değer = 6
 2. Thread 2 okur: Beklenen Değer = 5, Yeni Değer = 6

 CPU DONANIM SEVİYESİNDE ATOMİK TALİMAT: `CMPXCHG` (Compare and Exchange)

 Thread 1 CPU'ya sorar: "Bellekteki değer hala 5 mi?"
 CPU: "Evet, hala 5. Değeri 6 yapıyorum!" ──► [ BAŞARILI (CAS SUCCESS) ]

 Thread 2 CPU'ya sorar: "Bellekteki değer hala 5 mi?"
 CPU: "HAYIR! Değer artık 6 oldu!"        ──► [ BAŞARISIZ (CAS FAIL) ]

 Thread 2 pes etmez (Spin / Retry Loop):
 Yeniden okur: Beklenen = 6, Yeni = 7 ──► Tekrar dener ──► BAŞARILI!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir banko düşünün. Kilitli sistemde (synchronized) veznedar içeri tek bir müşteri alır, kapıyı kilitler, işi bitene kadar dışarıdaki 100 kişi yağmurda bekler. Kilitsiz sistemde (CAS / Atomic) ise kapı her zaman açıktır! Herkes vezneye koşar: 'Hesabımda 100 TL olduğunu biliyorum, onu 150 yap!' der. Veznedar bakar: 'Evet gerçekten 100 TL var, 150 yaptım (Başarılı).' İkinci kişi geldiğinde 'Hesabımda 100 TL vardı 120 yap' derse veznedar: 'Geç kaldın, para artık 150 oldu! Git sıranı tekrar hesapla gel!' der. Müşteri dışarıda beklemez, hemen güncel paraya göre tekrar dener!

🇬🇧 ENGLISH (ELI5)

Consider an amusement park turnstile. Pessimistic locking (synchronized) puts an armed guard at the gate: only one person enters, while 500 people wait frozen in the cold. Optimistic lock-free computing (CAS) removes the guard entirely! Everyone pushes forward: 'I observed the ticket counter was at 100; advance it to 101!' The hardware checks: 'Is it still 100? Yes! Flipping to 101 (Success).' If another person arrives claiming it was 100, hardware rejects: 'False! It is already 101! Recalculate and retry.' Nobody sleeps; losers spin and retry instantly!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CAS Mekanizması ve Donanım Desteği: 1) `CMPXCHG` Talimatı: x86 mimarisinde `lock cmpxchg` tek bir CPU saat çevriminde atomik olarak belleği kontrol eder ve eşitse yazar. Bu işlem işletim sistemi seviyesinde thread askıya alma (park/unpark) gerektirmez; kullanıcı modunda (user space) ultra hızlı gerçekleşir. 2) İyimser Kilitlenme (Optimistic Concurrency): Çatışmanın düşük olduğu sistemlerde CAS, geleneksel mutex kilitlerden 10 kat daha hızlıdır. 3) Spin-Wait Maliyeti: Yüksek çekişmede (high contention), onlarca thread aynı anda CAS denerse CPU döngüleri boşa harcanır (CPU spike) ve throughput düşer.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Hardware CAS Mechanics: 1) `cmpxchg` Assembly Primitive: AtomicInteger relies on `Unsafe` / `VarHandle.compareAndSet()`, executing the hardware-level `lock cmpxchg` instruction. It evaluates `if (memory == expected) { memory = update; return true; } else return false;` in a single un-interruptible silicon cycle. 2) Lock-Free Progress Guarantee: At least one thread is mathematically guaranteed to make forward progress on every CAS collision round, preventing priority inversions and deadlock. 3) Contention Penalty: Under extreme multi-threaded writes, CAS failure rates skyrocket, generating spin-wait CPU burn as threads continuously thrash on cache-line invalidations.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: AtomicInteger ile Thread-Safe Lock-Free Sayaç
import java.util.concurrent.atomic.AtomicInteger;

public class LockFreeCounter {
    private final AtomicInteger count = new AtomicInteger(0);

    public void increment() {
        // Dahili olarak compareAndSet döngüsü (CAS loop) işletir
        count.incrementAndGet();
    }

    public int get() {
        return count.get();
    }

    // Manuel CAS Döngüsü Örneği (Koşullu Güncelleme)
    public void multiplyIfLessThan(int multiplier, int limit) {
        int current;
        int next;
        do {
            current = count.get();
            if (current >= limit) return;
            next = current * multiplier;
        } while (!count.compareAndSet(current, next)); // Eşitse güncelle, değilse tekrar dene!
    }

    void main() {
        var counter = new LockFreeCounter();
        counter.increment();
        counter.multiplyIfLessThan(5, 10);
        IO.println("Lock-Free Son Değer: " + counter.get());
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'AtomicInteger kullanırsak thread'ler hiçbir zaman bloke olmaz mı?' Yanıt: 'Evet, işletim sistemi seviyesinde BLOCKED olmazlar; ANCAK yüksek çekişmede (High Contention) CAS sürekli başarısız olup döngüye (spin) gireceği için CPU kullanımı %100'e fırlayabilir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Interview Trap: 'Does AtomicInteger eliminate all thread blocking overhead?' Answer: 'Threads never enter the OS BLOCKED state; however, under extreme contention, repeated CAS failures cause aggressive spinning, spiking CPU utilization to 100% while starving other tasks.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- CAS (Compare-And-Swap), CPU donanım seviyesinde atomik kontrol ve yazma yapar.
- Kilit kullanılmadığı için Deadlock riski sıfırdır.
- Aşırı çekişmede CAS döngüleri CPU tüketimini tavan yaptırabilir.

🎯 Key Takeaways for the Interview

- CAS (Compare-And-Swap) executes atomic evaluate-and-set in hardware silicon via CMPXCHG.
- Lock-free algorithms carry zero risk of mutual deadlocks.
- Severe contention causes CAS spin-loops to burn excessive CPU cycles.